Chapter 2

Random Variable Generation

Contents

2.1 Introduction and motivation 29
2.2 Uniform pseudo-random number generation 30
2.2.1 Pseudo-random number generators 30
2.2.2 A famous failure and modern generators 31
2.3 The inverse transform method 31
2.3.1 Time between clicks 31
2.3.2 The inverse transform theorem 32
2.3.3 A new example: log-odds from uniforms 33
2.4 Transformation methods 33
2.4.1 Normal draws for the log-odds prior 33
2.4.2 Change of variables 33
2.4.3 Computing a posterior from scratch 35
2.5 The accept–reject algorithm 35
2.5.1 Sampling the non-conjugate posterior 35
2.5.2 The accept–reject theorem 35
2.5.3 When accept–reject becomes expensive 37
2.6 Sampling from standard distributions 37

2.1 Introduction and motivation

We observed in Chapter 1 that the Monte Carlo estimator (1.13) turns any posterior expectation into an average, provided that we can draw values $\theta^{(1)}, \ldots, \theta^{(N)}$ from the posterior.
Using our campaign example described in Section 1.2.1, we obtained 100 000 draws from $\text{Beta}(14, 226)$ with a single call, and estimated $\Pr(\theta > 0.05 \,|\, \mathbf{y}) \approx 0.69$. This chapter opens that call, and then, we ask and answer three different questions using the campaign example.

1. Where does randomness come from in a deterministic machine? (Section 2.2)
2. How do we turn uniform numbers into draws from a distribution with a known distribution function? (Sections 2.3 and 2.4)
3. How do we draw from a density that we know only up to a constant, such as the non-conjugate posterior of Example 1.4? (Section 2.5)

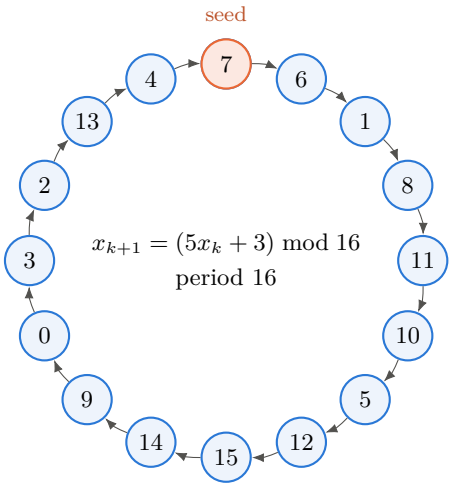


Figure 2.1: The toy linear congruential generator $x_{k+1} = (5x_k + 3) \bmod 16$ started at the seed $x_0 = 7$ visits all 16 states before repeating: its period is 16.

By the end of this chapter, we will have rebuilt the Chapter 1 posterior from uniform numbers alone (Figure 2.5), and we will have drawn exact samples from a posterior that has no closed form. We will also see why the second of these tools does not scale, which motivates Chapters 3, 5 and 6.

2.2 Uniform pseudo-random number generation

Example 2.1 (A toy generator)**.** A computer follows instructions, and it cannot flip a coin. What it can do, however, is produce a long sequence of numbers that behaves as if it were random. The simplest recipe multiplies, adds, and keeps a remainder. Take the integers $m = 16$, $a = 5$, and $c = 3$, start from $x_0 = 7$, and repeat by computing the next value through the following definition

$$x_{k+1} = (a\, x_k + c) \bmod m,$$

where $r \bmod m$ is the remainder of the division of r by m. The first steps are $x_1 = (5 \cdot 7 + 3) \bmod 16 = 38 \bmod 16 = 6$, then $x_2 = 33 \bmod 16 = 1$, and the sequence continues

$$6,\ 1,\ 8,\ 11,\ 10,\ 5,\ 12,\ 15,\ 14,\ 9,\ 0,\ 3,\ 2,\ 13,\ 4,\ 7,\ 6,\ 1,\ \ldots$$

Dividing by m gives numbers in the interval $[0, 1)$: $u_1 = 6/16 = 0.375$, $u_2 = 0.0625$, $u_3 = 0.5$, $u_4 = 0.6875$, ... After 16 steps, the generator is back at $x_0 = 7$, and the sequence repeats (Figure 2.1).

Each of the 16 possible values appears exactly once per cycle. After 16 steps, the generator is back at $x_0 = 7$, and the sequence repeats (Figure 2.1), and with $c = 2$ it is $5, 11, 9, 15, 13, 3, 1, 7$ (length 8, odd numbers only).

2.2.1 Pseudo-random number generators

Definition 2.1 (Pseudo-random number generator)**.** A **pseudo-random number generator (PRNG)** is a deterministic recursion $x_{k+1} = T(x_k)$ on a finite set of states, together with an output function $u_k = G(x_k) \in [0, 1)$. The initial state x_0 is the **seed**. The **period** is the number of steps after which the sequence of states repeats.

Because the set of states is finite, every PRNG is eventually periodic. A good generator has a period far longer than any simulation will use, and outputs $u_1, u_2, \ldots, u_n$ that pass statistical tests of the hypothesis "independent draws from $\mathcal{U}(0,1)$", the uniform distribution on $(0,1)$. It should also be fast and **reproducible**. In other words, the same seed produces the same sequence, making the simulation reproducible.

Definition 2.2 (Linear congruential generator)**.** A **linear congruential generator (LCG)** with modulus m, multiplier a and increment c is the recursion $x_{k+1} = (a\,x_k + c) \bmod m$ on $\{0, 1, \ldots, m-1\}$, with output $u_k = x_k/m$.

The period of an LCG is at most m. The following classical result, due to Hull and Dobell (1962), tells us when this maximum is attained.

Theorem 2.1 (Full period of an LCG)**.** *An LCG with $c \neq 0$ has period m for every seed if and only if (i) c and m have no common divisor other than 1; (ii) $a-1$ is divisible by every prime factor of m; and (iii) $a-1$ is divisible by 4 if m is divisible by 4.*

For the toy generator (Example 2.1), $m = 16 = 2^4$: $c = 3$ is odd, so (i) holds; $a - 1 = 4$ is divisible by 2 and by 4, so (ii) and (iii) hold, and the period is 16. With $a = 3$, $a - 1 = 2$ is not divisible by 4 and the period drops to 8; with $c = 2$, condition (i) fails.

2.2.2 A famous failure and modern generators

A long period is necessary but not sufficient. RANDU Press et al. (1992), an LCG with $a = 65539$, $c = 0$ and $m = 2^{31}$, was the standard generator on IBM computers in the 1960s and 1970s. Because $a = 2^{16} + 3$, squaring gives $a^2 \equiv 6a - 9 \pmod{2^{31}}$, and consecutive outputs satisfy

$$x_{k+2} = 6\,x_{k+1} - 9\,x_k \pmod{2^{31}}.$$

Consequently $9u_k - 6u_{k+1} + u_{k+2}$ is always an integer, and every triple (u_k, u_{k+1}, u_{k+2}) lies on one of only 15 parallel planes of the unit cube. A simulation that uses triples of RANDU numbers, for example, to draw three channel effects at once, explores only these planes. Such defects led to the systematic statistical testing of generators.

Modern libraries use generators with enormous periods and excellent high-dimensional uniformity. For instance, NumPy's default, PCG64, has period $2^{128} \approx 3.4 \times 10^{38}$. In Python, `np.random.default_rng(seed=42)` always produces $0.774, 0.439, 0.859, \ldots$, whereas the seed 43 produces an unrelated sequence.

2.3 The inverse transform method

2.3.1 Time between clicks

Clicks on the campaign's advertisement arrive at a rate of $\lambda = 2$ per minute. A standard model for the time X (in minutes) between two consecutive clicks is the exponential distribution with rate λ, whose **cumulative distribution function (CDF)** is

$$F(x) = \Pr(X \le x) = 1 - e^{-\lambda x}, \qquad x \ge 0.$$

F increases continuously from 0 to 1, so for every $u \in (0,1)$ the equation $u = F(x)$ has exactly one solution:

$$u = 1 - e^{-\lambda x} \iff x = -\frac{\log(1-u)}{\lambda} = F^{-1}(u).$$

The recipe is therefore: draw $U \sim \mathcal{U}(0,1)$ and return $X = -\log(1-U)/\lambda$.

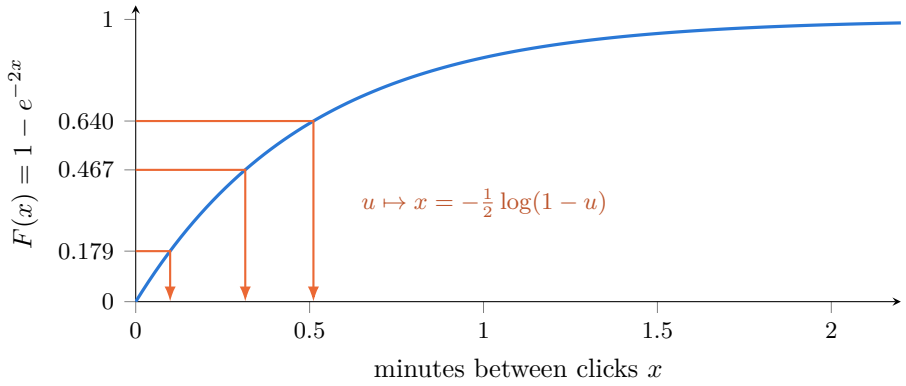


Figure 2.2: Inverse transform for the time between clicks, $X \sim \text{Exponential}(2)$. A uniform number u on the vertical axis is sent horizontally to the CDF and then down to $x = F^{-1}(u)$.

Algorithm 1: Inverse transform sampling

Input: a quantile function F^-, a uniform generator, a sample size N
Output: $X_1, \dots, X_N$ i.i.d. with CDF F

```
for i = 1, ..., N do
    draw U_i ~ U(0,1);
    X_i ← F⁻(U_i);
```

For example, the uniform numbers 0.179, 0.640 and 0.467 become waiting times of 0.099, 0.511 and 0.315 minutes (Figure 2.2). With 100 000 draws, the sample mean is 0.4976 and the sample variance 0.2486, against the exact values $1/\lambda = 0.5$ and $1/\lambda^2 = 0.25$.

2.3.2 The inverse transform theorem

Not every CDF can be inverted in the usual sense. A discrete distribution has a cumulative distribution function with jumps and flat parts. The following definition covers every case.

Definition 2.3 (Generalized inverse)**.** The **generalized inverse** (or quantile function) of a cumulative distribution function F is

$$F^-(u) = \inf\{x \in \mathbb{R} : F(x) \geq u\}, \qquad u \in (0,1),$$

the smallest x at which F reaches the level u. When F is continuous and strictly increasing, $F^- = F^{-1}$.

Theorem 2.2 (Inverse transform)**.** *Let F be any CDF and $U \sim \mathcal{U}(0,1)$. Then $X = F^-(U)$ has CDF F.*

Proof. Because F is non-decreasing and right-continuous, for every $u \in (0,1)$ and $x \in \mathbb{R}$ we have $F^-(u) \leq x \iff u \leq F(x)$. Hence $\Pr(X \leq x) = \Pr(F^-(U) \leq x) = \Pr(U \leq F(x)) = F(x)$, the last equality because $\Pr(U \leq t) = t$ for $t \in [0,1]$. □

For a discrete variable taking values $v_1 < v_2 < \dots$ with probabilities $p_1, p_2, \dots$, the generalized inverse is a table look-up: return the first v_k such that $p_1 + \cdots + p_k \geq u$. Geometrically, the interval $(0,1)$ is cut into consecutive pieces of lengths $p_1, p_2, \dots$, and u falls in piece k with probability p_k.

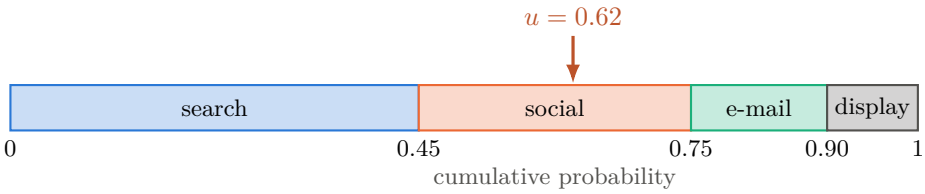


Figure 2.3: Discrete inverse transform: the unit interval is cut into pieces whose lengths are the channel probabilities. The uniform number $u = 0.62$ selects "social".

Example 2.2 (Which channel brought the visitor?)**.** The last channel touched before a click is search, social media, e-mail, or display advertising, with probabilities of 0.45, 0.30, 0.15, and 0.10, respectively. The cumulative probabilities are 0.45, 0.75, 0.90 and 1. A uniform number $u = 0.62$ lies between 0.45 and 0.75, so the visitor came from social media (Figure 2.3). With 100 000 draws, the simulated frequencies are 0.4519, 0.3012, 0.1474 and 0.0995.

2.3.3 A new example: log-odds from uniforms

Example 2.3 (Logistic distribution)**.** In Example 1.4 we used the logistic function $\sigma(\eta) = 1/(1 + e^{-\eta})$ to map log-odds to probabilities. The same function is the CDF of the **logistic distribution**. Solving $u = \sigma(x)$ gives $x = \log\{u/(1-u)\} = \text{logit}(u)$: the logit of a uniform number is a logistic random variable. In the simulation, 100 000 such draws have mean -0.002 and variance 3.2900, against the exact values 0 and $\pi^2/3 \approx 3.2899$.

Remark 2.1 (Checking a sampler)**.** A histogram against the target density is a useful visual check. A formal check is the **Kolmogorov–Smirnov test**, which compares the empirical CDF of the draws with F; a small p-value (say below 0.01) signals a mismatch. The exponential and logistic samplers above yield p-values of 0.145 and 0.591, respectively: no evidence against them.

The inverse transform needs F^- in a usable form. For the Normal, Gamma, and Beta distributions, which we need all the time, no closed-form expressions exist. The next two sections describe some alternatives.

2.4 Transformation methods

2.4.1 Normal draws for the log-odds prior

The log-odds prior of Example 1.4 is $\eta \sim \mathcal{N}(\mu_0, \sigma_0^2)$ with $\mu_0 = \text{logit}(0.05) \approx -2.944$ and $\sigma_0 = 0.5$. The standard Normal CDF Φ has no closed-form inverse, but a change of coordinates solves the problem. Take two independent uniforms U_1, U_2 and set

$$R = \sqrt{-2\log U_1}, \qquad Z_1 = R\cos(2\pi U_2), \qquad Z_2 = R\sin(2\pi U_2). \tag{2.1}$$

This is the **Box–Muller** transform (Figure 2.4). A random radius and a random angle define a point in the plane whose two coordinates turn out to be independent $\mathcal{N}(0,1)$ variables (Proposition 2.2). Then $\eta = \mu_0 + \sigma_0 Z$ has the required prior.

2.4.2 Change of variables

The general principle behind (2.1) is the change-of-variables formula for densities.

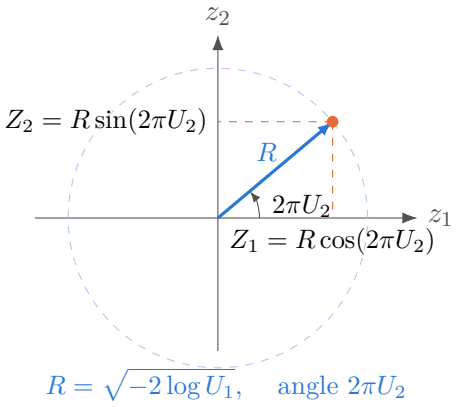


Figure 2.4: The Box–Muller transform. The radius R and the angle $2\pi U_2$ are built from two independent uniforms; the coordinates (Z_1, Z_2) of the point are independent standard Normal variables.

Proposition 2.1 (Change of variables)**.** *Let X be a random vector in $\mathbb{R}^d$ with density f_X, and let $T : \mathbb{R}^d \to \mathbb{R}^d$ be one-to-one and differentiable with a differentiable inverse on the region where $f_X > 0$. Then $Y = T(X)$ has density*

$$f_Y(y) = f_X(T^{-1}(y))\,\left|\det J_{T^{-1}}(y)\right|,$$

*where $J_{T^{-1}}(y)$ is the $d \times d$ matrix of partial derivatives of T^{-1} at y (the **Jacobian**) and* det *its determinant.*

In one dimension with T increasing, the formula reads $f_Y(y) = f_X(T^{-1}(y))\,\dfrac{\mathrm{d}}{\mathrm{d}y}T^{-1}(y)$. For instance, $Y = \mu + \sigma X$ with $\sigma > 0$ has density $f_X((y-\mu)/\sigma)/\sigma$: this is how every **location–scale** family (e.g., Normal, logistic, among others) is obtained from its standard member.

Proposition 2.2 (Box–Muller)**.** *If $U_1, U_2 \overset{iid}{\sim} \mathcal{U}(0,1)$, then Z_1, Z_2 defined in* (2.1) *are independent $\mathcal{N}(0,1)$.*

Sketch. $R^2 = -2\log U_1$ is exponential with mean 2 (inverse transform with $\lambda = 1/2$), and the angle $\Phi = 2\pi U_2$ is uniform on $[0, 2\pi)$, independent of R. Their joint density is $\dfrac{1}{2}e^{-r^2/2} \cdot 2r \cdot \dfrac{1}{2\pi}$ in (r, ϕ). The map $(r, \phi) \mapsto (z_1, z_2) = (r\cos\phi, r\sin\phi)$ has inverse Jacobian determinant $1/r$, so by Proposition 2.1 the density of (Z_1, Z_2) is $\dfrac{1}{2\pi}e^{-(z_1^2+z_2^2)/2} = \varphi(z_1)\varphi(z_2)$, the product of two standard Normal densities φ. □

Sums and ratios of simple variables give further families (Robert and Casella, 2004; Ross, 2014):

- if $E_1, \ldots, E_k$ are i.i.d. exponential with mean β, then $E_1 + \cdots + E_k \sim \text{Gamma}(k, \beta)$ (shape k, scale β);
- if $Z_1, \ldots, Z_k \overset{\text{iid}}{\sim} \mathcal{N}(0,1)$, then $Z_1^2 + \cdots + Z_k^2 \sim \chi^2_k$, the chi-square distribution with k degrees of freedom;
- if $G_1 \sim \text{Gamma}(a, 1)$ and $G_2 \sim \text{Gamma}(b, 1)$ are independent, then $G_1/(G_1 + G_2) \sim \text{Beta}(a, b)$.

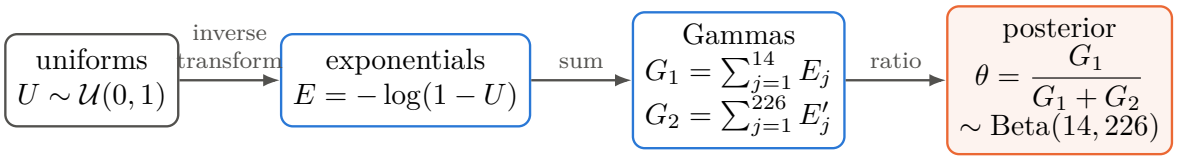


Figure 2.5: The posterior Beta(14, 226) built from uniform numbers: inverse transform, then sums, then a ratio.

2.4.3 Computing a posterior from scratch

Example 2.4 (Purchase amounts and the conversion rate)**.** The purchase amounts of Example 1.3 follow Gamma(2, 30): the sum of two independent exponential amounts with mean 30 euros. Built this way, 100 000 simulated amounts have a mean of 59.92 euros and a fraction of 0.1542 above 100 euros.

The posterior Beta(14, 226) (Section 1.2.1) has integer parameters, so the three facts above chain together (Figure 2.5): 14 exponentials give $G_1 \sim \text{Gamma}(14, 1)$, 226 exponentials give $G_2 \sim \text{Gamma}(226, 1)$, and $\theta = G_1/(G_1 + G_2) \sim \text{Beta}(14, 226)$. With 20 000 posterior draws (which consume 4.8 million uniform numbers) we obtain a posterior mean of 0.0585 (exact 0.0583) and $\Pr(\theta > 0.05 \,|\, \mathbf{y}) \approx 0.6888 \pm 0.0064$ (exact 0.6897). The Chapter 1 answer is now reproduced from uniform numbers only.

This construction is exact but wasteful, as it required 240 uniforms per draw. Modern implementations use more sophisticated algorithms, as the one described in Section 2.6, many of which rely on the accept–reject algorithm described in the next section.

2.5 The accept–reject algorithm

2.5.1 Sampling the non-conjugate posterior

We return to the log-odds model of Example 1.4. Its unnormalized posterior is

$$\tilde{\pi}(\eta \,|\, \mathbf{y}) = L(\eta)\, \pi(\eta), \qquad L(\eta) = \sigma(\eta)^s (1 - \sigma(\eta))^{n-s}, \qquad \pi(\eta) = \mathcal{N}(\eta;\ \mu_0, \sigma_0^2),$$

and neither the inverse transform nor a known transformation applies. But we can sample from the prior (Section 2.4.1), and the likelihood is bounded by its maximum $L_{\max} = L(\hat{\eta}) \approx 1.93 \times 10^{-20}$, reached at the MLE $\hat{\theta} = 0.06$. Hence

$$\tilde{\pi}(\eta \,|\, \mathbf{y}) \le L_{\max}\, \pi(\eta) \qquad \text{for every } \eta :$$

the scaled prior is an **envelope** that lies above the target everywhere (Figure 2.6). This suggests a two-step recipe: propose η from the prior, then keep it with probability $L(\eta)/L_{\max}$, that is, with a probability equal to the height of the target divided by the height of the envelope. Proposals in regions where the likelihood is high are almost always kept; proposals where the data are unlikely are usually discarded.

In our example, of 20 000 proposals, 9 884 are accepted, giving us an acceptance rate of 0.494. The accepted values give $\mathbb{E}[\theta \,|\, \mathbf{y}] \approx 0.0576$ and $\Pr(\theta > 0.05 \,|\, \mathbf{y}) \approx 0.688$, matching the quadrature values 0.0576 and 0.685 of Example 1.4 within Monte Carlo error. We have exact draws from a posterior without a closed form, and without computing its normalizing constant.

2.5.2 The accept–reject theorem

Theorem 2.3 (Accept–reject)**.** *Let $\tilde{f} \ge 0$ with $Z = \int \tilde{f}(x)\,\mathrm{d}x \in (0, \infty)$, let g be a density, and let M satisfy $\tilde{f}(x) \le M g(x)$ for all x. Then the output of the accept-reject algorithm (Al-*

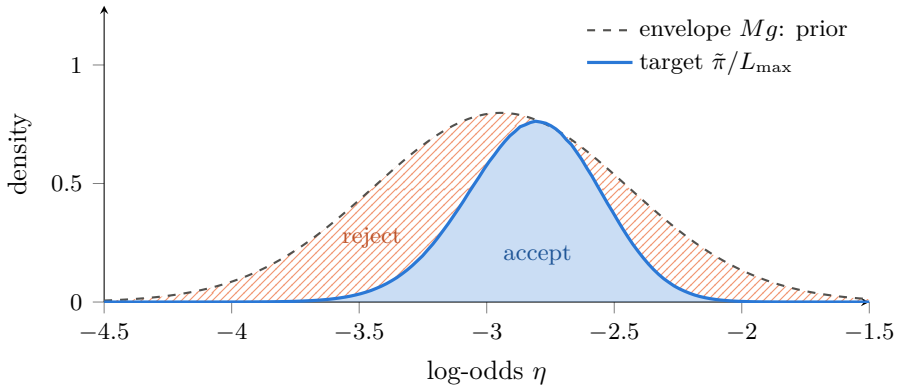


Figure 2.6: Accept–reject for the log-odds posterior with the prior as proposal. A proposal η is accepted with probability equal to the ratio of the solid curve (target) to the dashed curve (envelope) at η; the shaded area is accepted, the hatched area rejected.

Algorithm 2: Accept–reject

Input: an unnormalized target $\tilde{f}$, a proposal density g that we can sample, a constant M with $\tilde{f}(x) \le M g(x)$ for all x

Output: one draw X with density $f = \tilde{f}/Z$, where $Z = \int \tilde{f}$

repeat
  draw $Y \sim g$ and $U \sim \mathcal{U}(0,1)$, independently;
until $U \le \tilde{f}(Y)/(Mg(Y))$;
return $X \leftarrow Y$;

gorithm 2) has density $f = \tilde{f}/Z$. Each iteration accepts with probability Z/M, so the number of proposals needed per accepted draw is geometric with mean M/Z.

Proof. In one iteration, for any set A,

$$\Pr(Y \in A,\ \text{accept}) = \int_A g(y)\,\frac{\tilde{f}(y)}{Mg(y)}\,\mathrm{d}y = \frac{1}{M}\int_A \tilde{f}(y)\,\mathrm{d}y.$$

With $A = \mathbb{R}^d$, the acceptance probability is Z/M. Dividing, $\Pr(Y \in A \,|\, \text{accept}) = \int_A \tilde{f}(y)\,\mathrm{d}y/Z = \int_A f(y)\,\mathrm{d}y$. Iterations are independent, so the returned value is distributed as Y given acceptance, and the number of iterations is geometric with success probability Z/M. □

Three consequences guide the use of the method.

- *Only $\tilde{f}$ is needed.* The normalizing constant Z is never computed; in Bayesian applications $\tilde{f} = p(\mathbf{y}\,|\,\theta)\pi(\theta)$ and $Z = p(\mathbf{y})$. As a by-product, the acceptance rate estimates Z/M. In the campaign, $Z/M = p(\mathbf{y})/L_{\max} = 9.53 \times 10^{-21}/1.93 \times 10^{-20} = 0.493$, and we observed 0.494.
- *The best constant is $M = \sup_x \tilde{f}(x)/g(x)$.* A larger M is valid but wastes proposals.
- *The proposal must have heavier tails than the target.* If $\tilde{f}/g$ is unbounded, no valid M exists. A Normal proposal cannot be used for a target whose tails decay polynomially (Example 2.5).

When the prior is used as the proposal, the recipe of Section 2.5.1 is exactly Algorithm 2 with $g = \pi$, $M = L_{\max}$, and acceptance ratio $L(\eta)/L_{\max}$.

2.5.3 When accept–reject becomes expensive

Example 2.5 (The cost of accept–reject)**.**

More data. Keep the conversion rate at 6% but increase the number of visitors. The posterior narrows while the prior does not, and fewer prior proposals fall where the posterior is:

Visitors n	**Acceptance rate** Z/M	**Proposals per accepted draw**
200	0.493	2.0
2 000	0.173	5.8
20 000	0.055	18.1

More dimensions. Let the target be $\mathcal{N}(0, I_d)$, as in Example 1.6, and the proposal $\mathcal{N}(0, 1.5^2 I_d)$, slightly wider in every direction. The ratio of densities is maximal at the origin, where it equals $M = 1.5^d$, so the acceptance rate is 1.5^{-d}:

d	1	2	5	10	20	50
acceptance rate	0.667	0.444	0.132	0.017	3.0×10^{-4}	1.6×10^{-9}

A marketing mix model with 50 coefficients would need about 600 million proposals per accepted draw.

A poor envelope. Sampling Beta$(14, 226)$ with a uniform proposal on $(0, 1)$ requires M equal to the maximum of the density, 27.0, reached at the mode 0.0546. Only $1/27 = 3.7\%$ of proposals are accepted (simulated: 3.67%), because the proposal spreads its mass over $(0, 1)$ while the posterior lives in $(0.02, 0.11)$.

The accept–reject algorithm is exact and simple, and it is an excellent tool in low dimensions with a well-matched proposal. But its cost is tied to how well a single envelope covers the target, and in high dimensions. That cost grows exponentially (i.e., the corners of Figure 1.7). Chapter 3 avoids discarding draws by reweighting them (importance sampling), and Chapters 5 and 6 build proposals that adapt to where the chain currently is.

2.6 Sampling from standard distributions

In practice, we rarely implement the methods above for standard distributions, as different libraries (e.g., NumPy) provide carefully optimized versions. It is still useful to know what they do, because the same ideas are used to design samplers for new models. For instance, Table 2.1 summarizes the algorithms behind NumPy's `Generator` methods.

Correlated channel effects. Suppose that past analyses suggest effects of TV and search on the log-odds of conversion with means 0.3 and 0.5, standard deviations 0.2 and 0.1, and correlation -0.6 (when a budget moves from one channel to the other, their estimated effects trade off). We want draws of the pair $\beta = (\beta_{\text{TV}}, \beta_{\text{search}})$ from the bivariate Normal with mean $\mu = (0.3, 0.5)$ and covariance matrix

$$\Sigma = \begin{pmatrix} 0.2^2 & -0.6 \cdot 0.2 \cdot 0.1 \\ -0.6 \cdot 0.2 \cdot 0.1 & 0.1^2 \end{pmatrix} = \begin{pmatrix} 0.04 & -0.012 \\ -0.012 & 0.01 \end{pmatrix}.$$

Table 2.1: Algorithms used by NumPy's `Generator` for common distributions.

Distribution	Method	Algorithm
Uniform	`random`	PCG64 bits converted to a double
Exponential, Normal	`exponential`, `normal`	ziggurat (a fast accept–reject)
Gamma	`gamma`	Marsaglia–Tsang accept–reject
Beta	`beta`	ratio of Gammas (Jöhnk for small shapes)
Binomial, Poisson	`binomial`, `poisson`	inversion (small mean), accept–reject
Multivariate Normal	`multivariate_normal`	$\mu + LZ$ with a matrix square root L

The Cholesky construction.

Proposition 2.3 (Multivariate Normal)**.** *Let Σ be a symmetric positive definite $d \times d$ matrix and L a matrix with $LL^\top = \Sigma$; the **Cholesky factor** is the unique such L that is lower triangular with positive diagonal. If $Z = (Z_1, \dots, Z_d)$ has i.i.d. $\mathcal{N}(0,1)$ components, then $\mu + LZ \sim \mathcal{N}(\mu, \Sigma)$.*

Proof. A linear transformation of a Gaussian vector is Gaussian, with mean $\mu + L\,\mathbb{E}[Z] = \mu$ and covariance $L\ \mathrm{Cov}(Z)\ L^\top = LI_dL^\top = \Sigma$. □

Back to the example. Here $L = \begin{pmatrix} 0.2 & 0 \\ -0.06 & 0.08 \end{pmatrix}$ (check: $LL^\top = \Sigma$). With 100 000 pairs of Box–Muller normals, the simulated effects have means $(0.301, 0.500)$, standard deviations $(0.200, 0.100)$ and correlation -0.601. As a by-product, the probability that search has the larger effect is $\Pr(\beta_{\text{search}} > \beta_{\text{TV}}) \approx 0.767$.

Summary

Takeaways

- All simulations start from a pseudo-random generator: a deterministic recursion with a seed and a period. Reproducibility comes from fixing the seed; quality comes from a long period and good uniformity in several dimensions.
- Inverse transform: $X = F^-(U)$ has CDF F. It works for every distribution whose quantile function we can compute, including discrete ones.
- Transformations (change of variables, sums, ratios) produce the Normal, Gamma, chi-square, and Beta families from simple building blocks.
- Accept–reject samples any density known up to a constant, given an envelope $Mg \geq \tilde{f}$. It accepts a fraction Z/M of proposals, which shrinks as the data grows and increases exponentially with the dimension.

Further reading

Chapter 2 of Robert and Casella (2004) covers uniform generators, the inverse transform, transformations, and accept–reject in depth. Chapter 1 of Gamerman and Lopes (2006) gives a concise treatment oriented to Bayesian computation, and the chapter on simulation of Ross (2014) presents the generation of standard distributions with many worked examples.